

Ide proyek ini adalah memprediksi rating 1–5 dari review pelanggan secara otomatis menggunakan **DistilBERT**, model pretrained yang sudah memahami konteks bahasa Inggris. Setiap review diubah menjadi token numerik dan diproses melalui classifier, sementara bobot BERT dibekukan agar pelatihan lebih cepat. Untuk menangani ketidakseimbangan rating, digunakan class weights dalam loss function. Dengan pendekatan ini, teks review diubah menjadi prediksi rating, memungkinkan analisis sentimen pelanggan secara efisien dan kontekstual.

```
# ===== 1. Upload File CSV =====
from google.colab import files
import pandas as pd

uploaded = files.upload()
```

[Choose Files](#) review.csv
review.csv(text/csv) - 1208917 bytes, last modified: 11/22/2025 - 100% done
 Saving review.csv to review.csv

1. Data Loading & Cleaning (Preprocessing)

Karena langkah-langkah mencakup:

- membaca file CSV → data loading
- menghapus kolom tak penting → data cleaning
- menghapus baris kosong → preprocessing
- membuang baris tanpa review → data filtering
- reset index → formatting

```
import pandas as pd
import numpy as np

# Ambil nama file yang di-upload
filename = list(uploaded.keys())[0]
print("File uploaded:", filename)

# ===== 2. Baca CSV =====
df = pd.read_csv(filename, sep=';')
# 1. Hapus semua kolom Unnamed yang isinya NaN semua
df = df.loc[:, ~df.columns.str.contains('^Unnamed')]

# 2. Hapus baris yang semua kolomnya kosong
df = df.dropna(how="all")

# 3. Jika ada baris yang review_text kosong → hapus juga
df = df.dropna(subset=["review_text"])

# 4. Reset index (opsional)
df = df.reset_index(drop=True)

df.head()
```

File uploaded: review.csv

	meal_id	rating	review_text	
0	B001E4KFG0	5.0	I have bought several of the Vitality canned d...	
1	B00813GRG4	1.0	Product arrived labeled as Jumbo Salted Peanut...	
2	B000LQOCH0	4.0	This is a confection that has been around a fe...	
3	B000UA0QIQ	2.0	If you are looking for the secret ingredient i...	
4	B006K2ZZ7K	5.0	Great taffy at a great price. There was a wid...	

Next steps: [Generate code with df](#) [New interactive sheet](#)

2. Cek Rating Proportion

- melihat komposisi atau proporsi rating 1–5
- memahami apakah rating lebih banyak positif atau negatif
- bentuk paling dasar dari exploratory data analysis

```
print(df['rating'].value_counts(normalize=True) * 100)
```

```
rating
5.0    62.838969
4.0    13.659190
1.0     9.507867
3.0     8.101774
2.0     5.892200
Name: proportion, dtype: float64
```

Lebih dari 62% review adalah rating 5, dan rating menengah (2–4) jauh lebih sedikit.

Ini menyebabkan:

- Model cenderung belajar pola sederhana: positif → rating 5, negatif → rating 1
- Rating 2, 3, dan 4 muncul sangat jarang → model tidak terlatih mengenal pola bahasa untuk rating tersebut.
- Hasil prediksi model yang sering terjadi: 1 atau 5, jarang memunculkan 2–4.

Model menjadi bias ke rating 5 karena 62% data adalah rating 5.

ML

Library yang digunakan:

1. Library untuk manipulasi data: pandas, numpy
2. Library untuk deep learning:
 - torch → Framework utama untuk membangun model neural network, menghitung loss, backward pass, dan optimasi bobot.
 - torch.utils.data.Dataset & DataLoader → Membuat dataset custom dan batching/shuffling otomatis untuk training/testing.
 - torch.optim.AdamW → Optimizer untuk memperbarui bobot model dengan weight decay, membantu convergence lebih stabil.
 - torch.nn → Menyediakan loss function (CrossEntropyLoss) dan layer neural network.

Untuk membangun, melatih, dan mengevaluasi model klasifikasi berbasis deep learning.

3. Library untuk NLP pretrained model
 - transformers (DistilBertTokenizerFast & DistilBertForSequenceClassification)

Tokenizer → Mengubah teks menjadi token numerik yang bisa dipahami BERT.

Model → DistilBERT + classifier linear untuk klasifikasi rating 1–5.

- get_linear_schedule_with_warmup → Scheduler untuk mengatur learning rate saat training.

Memanfaatkan model pretrained agar bisa memahami konteks bahasa secara cepat tanpa melatih dari nol, sehingga prediksi rating lebih akurat.

```
!pip install transformers datasets torch tqdm scikit-learn -q
```

Show hidden output

```
import pandas as pd
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader
from torch.optim import AdamW
import torch.nn as nn
from tqdm import tqdm
from transformers import DistilBertTokenizerFast, DistilBertForSequenceClassification, get_linear_schedule_with_warmup
from sklearn.model_selection import train_test_split
```

```
from sklearn.utils.class_weight import compute_class_weight
```

1. Persiapan Data

- Cleaning data, menghapus semua baris yang memiliki nilai kosong
- Membuat kolom baru berupa label yang nilainya adalah rating - 1. Alasannya karena model klasifikasi nn.CrossEntropyLoss di PyTorch menerima label kelas sebagai integer mulai dari 0.

Misal ada 5 kelas → label harus 0, 1, 2, 3, 4.

- Split data: 80% untuk training dan 20% untuk testing. stratify memastikan distribusi label tetap sama antara train & test.
- Tokenizer mengubah teks menjadi token ID yang bisa dipahami model BERT

```
df = df.dropna(subset=["review_text", "rating"])
df["label"] = df["rating"] - 1
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42, stratify=df["label"])
tokenizer = DistilBertTokenizerFast.from_pretrained("distilbert-base-uncased")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 4.12kB/s]

vocab.txt: 100% 232k/232k [00:00<00:00, 14.6MB/s]

tokenizer.json: 100% 466k/466k [00:00<00:00, 29.6MB/s]

config.json: 100% 483/483 [00:00<00:00, 39.7kB/s]
```

2. Kelas Custom Dataset

Kode ini membuat kelas custom dataset untuk PyTorch agar data review bisa dipakai oleh model. Intinya: mengubah data mentah (teks dan label) menjadi format yang bisa dimengerti model BERT/DistilBERT.

```
class ReviewDataset(Dataset):
    def __init__(self, df, tokenizer, max_len=128):
        self.text = df["review_text"].tolist()
        self.labels = df["label"].tolist()
        self.tokenizer = tokenizer
        self.max_len = max_len

        #Apa yang dilakukan:
        #Menyimpan semua review (self.text) dan label (self.labels) dari dataframe.
        #Menyimpan tokenizer yang sudah disiapkan.
        #Menentukan panjang maksimum token (max_len=128) supaya input model selalu sama panjang.

    def __len__(self):
        return len(self.text)
        #Mengembalikan jumlah data dalam dataset.
        #PyTorch butuh ini supaya tahu berapa banyak batch yang bisa dibuat.

    def __getitem__(self, idx):
        enc = self.tokenizer(
            self.text[idx],
            truncation=True,
            padding="max_length",
            max_length=self.max_len,
            return_tensors="pt"
        )
        item = {key: val.squeeze(0) for key, val in enc.items()}
        item["labels"] = torch.tensor(self.labels[idx], dtype=torch.long)
        return item

# Apa yang dilakukan:
```

```
#Mengambil review ke-idx.

#Tokenisasi review → mengubah teks jadi angka (input IDs) dan attention mask.

#truncation=True → potong teks yang terlalu panjang

#padding="max_length" → tambahkan padding supaya semua input sama panjang

#return_tensors="pt" → ubah jadi tensor PyTorch

#Squeeze dimensi tambahan → agar bentuk tensor sesuai [seq_len].

#Tambahkan label ke dictionary → siap dipakai loss function.
```

3. Dataset & DataLoader

```
#Menentukan jumlah data yang diproses sekaligus oleh model.
batch_size = 32

#Apa yang dilakukan: Mengubah data mentah (review + label) menjadi format angka yang bisa dipahami model.
train_dataset = ReviewDataset(train_df, tokenizer)
test_dataset = ReviewDataset(test_df, tokenizer)

#Apa yang dilakukan: Membagi dataset train menjadi batch-batch berisi 32 review dan mengacak urutannya.
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size)
```

4. Menyiapkan Model DistilBERT dan Device untuk Training

menyiapkan model dan tempat komputasinya, serta menentukan bagian mana yang boleh dilatih.

```
#tempat di mana model dan data akan diproses.
device = "cuda" if torch.cuda.is_available() else "cpu"

#objek model yang siap dilatih.
model = DistilBertForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
    num_labels=5
).to(device)

# Freeze semua layer BERT → latih hanya classifier
for param in model.distilbert.parameters():
    param.requires_grad = False
```

```
model.safetensors: 100% 268M/268M [00:02<00:00, 170MB/s]

Some weights of DistilBertForSequenceClassification were not initialized from the model checkpoint at distilbert-base-uncased and are newly initialized from the random normal distribution. You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

5. Bobot kelas (class_weights)

Proses ini mengatur agar model tidak bias terhadap kelas yang lebih banyak. Bobot ini nanti akan dipakai di loss function (CrossEntropyLoss) supaya kesalahan di kelas minoritas dihitung lebih serius.

```
# bobot untuk setiap kelas rating (0-4) berdasarkan jumlahnya di dataset.
class_weights = compute_class_weight(
    class_weight="balanced",
    classes=np.array([0,1,2,3,4]),
    y=train_df["label"].values
)

#Mengubah bobot menjadi tensor PyTorch dan dikirim ke device (CPU/GPU) agar bisa dipakai dalam loss function.
class_weights = torch.tensor(class_weights, dtype=torch.float).to(device)
print("Class Weights:", class_weights)
```

```
Class Weights: tensor([2.1048, 3.3887, 2.4629, 1.4656, 0.3183])
```

```
#Loss function (loss_fn) → mengukur kesalahan prediksi model
loss_fn = nn.CrossEntropyLoss(weight=class_weights)
#Optimizer → menentukan bagaimana model belajar dari loss
optimizer = AdamW(model.parameters(), lr=2e-5)

#Jumlah epoch → kontrol lamanya model belajar.
num_epochs = 4

#Total steps → dipakai untuk scheduler learning rate.
total_steps = len(train_loader) * num_epochs

#Scheduler → mengatur kecepatan belajar optimizer selama training.
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=0,
    num_training_steps=total_steps
)
```

6. Training

Model belajar menebak rating review dengan cara melihat banyak batch data, mengukur kesalahan prediksi, dan memperbaiki dirinya sedikit demi sedikit hingga training selesai.

```
model.train()

for epoch in range(num_epochs):
    total_loss = 0
    loop = tqdm(train_loader, desc=f"Epoch {epoch+1}/{num_epochs}")

    for batch in loop:
        batch = {k: v.to(device) for k, v in batch.items()}
        labels = batch.pop("labels")

        optimizer.zero_grad()

        outputs = model(**batch).logits
        loss = loss_fn(outputs, labels)

        loss.backward()
        optimizer.step()
        scheduler.step()

        total_loss += loss.item()
        loop.set_postfix(loss=f"{loss.item():.4f}")

    print(f"Epoch {epoch+1} Loss: {total_loss/len(train_loader):.4f}")
```

```
Epoch 1/4: 100%|██████████| 75/75 [10:26<00:00, 8.36s/it, loss=1.5715]
Epoch 1 Loss: 1.6060
Epoch 2/4: 100%|██████████| 75/75 [09:52<00:00, 7.90s/it, loss=1.6216]
Epoch 2 Loss: 1.5945
Epoch 3/4: 100%|██████████| 75/75 [09:53<00:00, 7.91s/it, loss=1.5600]
Epoch 3 Loss: 1.5866
Epoch 4/4: 100%|██████████| 75/75 [09:56<00:00, 7.95s/it, loss=1.6169]Epoch 4 Loss: 1.5839
```

7. Evaluasi Model pada Test Set

Proses ini mengevaluasi seberapa baik model menebak rating review pada data yang belum pernah dilihat.

```
model.eval()

correct = 0
total = 0

with torch.no_grad():
    for batch in test_loader:
```

```

batch = {k: v.to(device) for k, v in batch.items()}
labels = batch.pop("labels")

logits = model(**batch).logits
preds = torch.argmax(logits, dim=1)

correct += (preds == labels).sum().item()
total += labels.size(0)

print("Test Accuracy:", correct / total)

```

Test Accuracy: 0.6137123745819398

8. Predict Rating

Fungsi ini memungkinkan kita memasukkan review baru dan mendapatkan prediksi rating dari model yang sudah dilatih, tanpa harus menjalankan loop training/evaluasi.

```

def predict_rating(text):
    enc = tokenizer(text, return_tensors="pt", padding="max_length", truncation=True, max_length=128)
    enc = {k: v.to(device) for k,v in enc.items()}

    with torch.no_grad():
        logits = model(**enc).logits
        pred = torch.argmax(logits).item()

    return pred + 1 # konversi ke rating 1-5

print(predict_rating("This tea is the most awful stuff I've ever put in my mouth. It has a chemical taste."))

```

5

9. Analisis

Beberapa faktor yang bisa menyebabkan akurasi masih rendah:

- Freezing BERT

Hanya classifier yang dilatih → model tidak belajar representasi teks spesifik data baru.

Bisa membantu komputasi tapi mengorbankan performa.

- Ukuran Dataset & Kualitas Data

Jika dataset kecil atau review mengandung banyak slang/sarkasme, model pretrained sulit menyesuaikan.

- Class Imbalance

Meskipun sudah pakai class_weights, kelas minoritas masih bisa sulit diprediksi.

- Hyperparameter

Max sequence length 128 mungkin terlalu pendek → banyak informasi hilang.

Batch size 32 & learning rate 2e-5 mungkin belum optimal untuk data ini.

Epoch 4 mungkin terlalu sedikit untuk convergence.

- Model DistilBERT

Lebih ringan → lebih cepat tapi kurang akurat dibanding BERT-base/full.

- Nature of Task

Sentiment review sering ambigu → rating 3 vs 4 bisa subjektif → akurasi maksimal untuk model sederhana biasanya 60–70%.

Close

10. Kesimpulan
Kode ini melakukan text classification rating review dengan Distil
PyTorch.

Alurnya: data cleaning → tokenisasi → dataset → dataloader → model

10. Kesimpulan

weights → training → evaluasi.

Akurasi 61% wajar karena model belum fully fine-tuned, data imbalance, dan kompleksitas bahasa alami.

Potensi improvement:

Unfreeze beberapa layer BERT (fine-tuning sebagian).

Tingkatkan max_len atau preprocessing teks (misal handle emojis/slang).

Tambah epoch atau gunakan learning rate scheduler berbeda.

Pakai model BERT-base atau RoBERTa untuk performa lebih tinggi.

Kode ini melakukan text classification rating review dengan DistilBERT + PyTorch.

Alurnya: data cleaning → tokenisasi → dataset → dataloader → model → class weights → training → evaluasi.

Akurasi 61% wajar karena model belum fully fine-tuned, data imbalance, dan kompleksitas bahasa alami.

Potensi improvement:

Unfreeze beberapa layer BERT (fine-tuning sebagian).

Tingkatkan max_len atau preprocessing teks (misal handle emojis/slang).

Tambah epoch atau gunakan learning rate scheduler berbeda.

Pakai model BERT-base atau RoBERTa untuk performa lebih tinggi.